Solving delay differential equations

A systematic guide: method of steps, characteristic equations, Lambert W, and what can (and cannot) be solved in closed form

1. What makes a DDE different

A delay differential equation (DDE) has the form $x'(t) = f(t, x(t), x(t-\tau))$, with a **history function** $x(t) = \varphi(t)$ given on the whole interval $[-\tau, 0]$. The state is therefore not a point in R^n but a function segment: the phase space is infinite-dimensional. Three consequences drive everything below:

- **Initial data is a function**, not a vector. Changing φ changes the solution.
- **Infinitely many characteristic roots** for linear problems, instead of n eigenvalues.
- **Discontinuities propagate.** If $x'(0^+) \neq \varphi'(0^-)$, the kink at $t = 0$ reappears (one derivative smoother) at $t = \tau, 2\tau, 3\tau, \ldots$ Solutions smooth out as t grows.

2. A systematic workflow

Whatever the equation, the following order of attack works well:

- **Classify the delay.** Constant (τ), several constants, time-dependent $\tau(t)$, state-dependent $\tau(x)$, distributed (an integral over the past), or neutral (delay on x', which is harder).
- **Find equilibria** and linearize around each one. This turns the nonlinear DDE into a linear constant-coefficient DDE.
- **Solve the characteristic equation** of the linearization (Lambert W when there is a single delay). The sign of the largest real part decides local stability; a root pair crossing the imaginary axis signals a Hopf bifurcation.
- **Try method of steps** for an exact piecewise solution, which is feasible when the right-hand side is simple enough to integrate repeatedly.
- **Otherwise go numeric** with a solver that tracks discontinuities and uses dense output for the delayed term.

3. Method of steps (exact, piecewise)

On $[0, \tau]$ the delayed term $x(t-\tau)$ is the known history φ, so the DDE becomes an ordinary ODE. Solve it, and the result is the known history on $[\tau, 2\tau]$, and so on. Each step is one ODE solve with a source term from the previous step.

Worked example. $x'(t) = -x(t-1)$, with $\varphi(t) = 1$ for $t \in [-1, 0]$ and $\tau = 1$.

Interval	Equation	Solution
$[0, 1]$	$x' = -\varphi(t-1) = -1,\ x(0) = 1$	$x(t) = 1 - t$
$[1, 2]$	$x' = -(1 - (t-1)) = t - 2,\ x(1) = 0$	$x(t) = (t - 1)(t - 3)/2$
$[2, 3]$	$x' = -(t-2)(t-4)/2,\ x(2) = -1/2$	$x(t) = -t^3/6 + 3t^2/2 - 4t + 17/6$

Notice the smoothing: x' jumps at $t = 0$ (0 on the left, -1 on the right), x' is continuous at $t = 1$ but x'' is not, and so on. The degree of the polynomial grows by one per step, so the symbolic expressions grow quickly: this is exact, but only practical for a handful of steps. SymPy automates it:

```
import sympy as sp

t = sp.symbols("t")
x = 1 - t                                   # solution on [0, 1]
for k in (1, 2):                            # steps [1, 2] and [2, 3]
    x = sp.integrate(-x.subs(t, t - 1), (t, k, t)) + x.subs(t, k)
    print(sp.expand(x))
```

4. Linear constant-coefficient DDEs: characteristic equation and Lambert W

For $x'(t) = a\,x(t) + b\,x(t-\tau)$, try $x = e^{\lambda t}$. This gives the characteristic equation

$$\lambda = a + b\,e^{-\lambda\tau}$$

It is transcendental, so it has infinitely many complex roots. Setting $s = \lambda - a$ and multiplying through gives $(s\tau)\,e^{s\tau} = b\tau\,e^{-a\tau}$, which is exactly the defining relation of the Lambert W function ($W(z)\,e^{W(z)} = z$). Hence all roots in closed form:

$$\lambda_k = a + W_k(\,b\tau\,e^{-a\tau}\,)\,/\,\tau, \quad k \in Z$$

where W_k are the branches of W. The general solution is $x(t) = \sum_k c_k\,e^{\lambda_k t}$, with coefficients c_k determined by the history φ (via residues of the Laplace transform, section 5). For real argument, the principal branch W_0 gives the **rightmost root**, which controls long-term behaviour.

Example. For $x' = -x(t-1)$ ($a = 0$, $b = -1$, $\tau = 1$), $\lambda_k = W_k(-1)$. Numerically (checked with SciPy):

k	$\lambda_k = W_k(-1)$
0	−0.3181 + 1.3372 i
1	−2.0623 + 7.5886 i
2	−2.6532 + 13.9492 i

All real parts are negative, so the solution decays with damped oscillations of angular frequency ≈ 1.34 and rate ≈ 0.32, consistent with the piecewise solution above, which oscillates through zero. For $a = 0$ the exact stability window is $-\pi/2 < b\tau < 0$.

```
import numpy as np
from scipy.special import lambertw

a, b, tau = 0.0, -1.0, 1.0
roots = [a + lambertw(b * tau * np.exp(-a * tau), k) / tau for k in range(-3, 4)]
rightmost = max(roots, key=lambda z: z.real)
print(rightmost)                        # (-0.318+1.337j): stable if real part < 0
```

5. Laplace transform and the role of the history

For the same linear equation, taking Laplace transforms (using the shift property) gives

$$X(s) = [\,\varphi(0) + b\,e^{-s\tau}\int_{-\tau}^{0} e^{-su}\,\varphi(u)\,du\,]\,/\,[\,s - a - b\,e^{-s\tau}\,]$$

The denominator is the characteristic function, so its zeros are the λ_k above. Inverting by residues gives $x(t) = \sum_k \mathrm{Res}_{s=\lambda_k}[\,X(s)\,e^{st}\,]$: the history enters only through the numerator, which is how it determines the coefficients c_k. This is exact but needs the full infinite root set; in practice a few dozen branches give an excellent approximation for $t > \tau$.

6. Nonlinear DDEs: linearize, then look for bifurcations

Closed forms essentially disappear once f is nonlinear, but the local picture stays tractable. Take the delayed logistic (Hutchinson) equation $x' = r\,x(t)\,(1 - x(t-\tau)/K)$ with equilibrium $x^* = K$. Writing $x = K + u$:

$$u'(t) = -r\,u(t-\tau) \;\Rightarrow\; \lambda = -r\,e^{-\lambda\tau} \;\Rightarrow\; \lambda_k = W_k(-r\tau)\,/\,\tau$$

The equilibrium is stable iff $r\tau < \pi/2$. At $r\tau = \pi/2$ the roots are $\lambda = \pm i\pi/(2\tau)$, a pair crossing the imaginary axis: a **Hopf bifurcation** to sustained oscillations of period $\approx 4\tau$. Beyond this point the nonlinear behaviour (amplitude, chaos in Mackey–Glass and similar models) requires numerics, or a normal-form / centre-manifold computation for the bifurcation amplitude.

7. Can we get a symbolic solution? A realistic summary

Class of problem	Closed form?	How
Linear, constant coefficients, one delay	Yes (infinite series)	Lambert W roots + residues / Laplace
Linear, several commensurate delays	Roots only numerically	Quasi-polynomial roots; Laplace + residues
Any DDE with simple polynomial/elementary f	Yes, on a few steps	Method of steps (SymPy)
Linear, time-varying coefficients	Rarely	Special cases via transformation or method of steps
Nonlinear (Hutchinson, Mackey–Glass, ...)	No, in general	Linearization, bifurcation analysis, numerics
State-dependent delay	Essentially never	Numerics (careful event handling)

Rule of thumb: **symbolic for the linear skeleton, numeric for the full nonlinear dynamics.** The Lambert W formula is the closest thing to a general closed form, and it works only for linear equations with one delay.

8. Numerical solution in practice

A good DDE solver is an ODE solver plus three extra ingredients:

- **History storage with dense output**: $x(t-\tau)$ is rarely on a mesh point, so the solver interpolates the past with the same order of accuracy as the integrator (continuous Runge–Kutta methods).
- **Discontinuity tracking**: step exactly onto $t = 0, \tau, 2\tau, \ldots$ (and their images for several or state-dependent delays); otherwise the error control silently degrades.
- **Step size $\leq \tau$** (or handling of overlapping steps) so that the delayed value is already known.

In Python, **JiTCDDE** does all three; here is the Mackey–Glass equation:

```
import numpy as np
from jitcdde import jitcdde, y, t

tau = 15.0
f = [0.2 * y(0, t - tau) / (1 + y(0, t - tau) ** 10) - 0.1 * y(0)]

dde = jitcdde(f)
dde.constant_past([1.0], time=0.0)         # history function phi(t) = 1 for t <= 0
dde.step_on_discontinuities()              # propagate the kink at t = 0, tau, 2 tau, ...
times = np.arange(dde.t, dde.t + 1000, 1.0)
xs = np.array([dde.integrate(T) for T in times])
```

Install with `uv add jitcdde` (needs a C compiler). Alternatives: **diffrax** (JAX; differentiable, GPU-friendly, delays handled more manually) and Julia's **DelayDiffEq.jl** for the most complete feature set, including state-dependent delays and neutral equations.

9. Common pitfalls

- **Ignoring the history.** Two DDEs with the same equation but different φ have different solutions; always specify φ on $[-\tau, 0]$.
- **Treating a DDE as an ODE of fixed size** by sampling $x(t-\tau)$ on a grid: this is the method of lines and loses accuracy at the kinks.
- **Trusting the real-part test with few roots.** Stability needs the rightmost root among infinitely many; for real arguments W_0 is it, but for several delays or complex coefficients, check numerically over many branches or use a dedicated quasi-polynomial root finder.
- **Large delays or stiffness.** Large τ makes the problem effectively high-dimensional (long-memory), and stiff terms need an implicit DDE solver.